

Backpropagation — Complete Cheatsheet

Forward pass → Compute loss → Backprop right-to-left → Update weights

STEP 0 · Network setup

Input x	2.0
Weight w	0.5
Bias b	0.8
True y	1.0

forward →

STEP 1 · Forward pass

$$\begin{aligned}z &= w \times x + b \\z &= 0.5 \times 2.0 + 0.8 \\z &= 1.8 \\y_{\text{hat}} &= \text{sigmoid}(z) \\y_{\text{hat}} &= 0.858\end{aligned}$$

predict →

STEP 2 · Compute loss (MSE)

$$\begin{aligned}L &= (y - y_{\text{hat}})^2 \\L &= (1.0 - 0.858)^2 \\L &= (0.142)^2 \\L &= 0.0202 \\ \text{Error} &= 0.142 \text{ (need to fix!)}\end{aligned}$$

■ BACKPROP direction — gradients flow right to left ■

STEP 3 · dL/dy_hat

$$\begin{aligned}dL/dy_{\text{hat}} &= -2(y - y_{\text{hat}}) \\&= -2 \times 0.142 \\&= -0.284\end{aligned}$$

STEP 4 · dL/dz (chain rule)

$$\begin{aligned}dy_{\text{hat}}/dz &= y_{\text{hat}}(1 - y_{\text{hat}}) \\&= 0.858 \times 0.142 = 0.1218 \\dL/dz &= -0.284 \times 0.1218 \\&= -0.0346\end{aligned}$$

STEP 5 · dL/dw (chain rule)

$$\begin{aligned}dz/dw &= x = 2.0 \\dL/dw &= dL/dz \times x \\&= -0.0346 \times 2.0 \\&= -0.0692\end{aligned}$$

STEP 6 · dL/db (chain rule)

$$\begin{aligned}dz/db &= 1 \\dL/db &= dL/dz \times 1 \\&= -0.0346 \times 1 \\&= -0.0346\end{aligned}$$

STEP 7 · Weight Update (Gradient Descent)

Weight w

$$\begin{aligned}0.500 &- 0.1 \times (-0.0692) \\&= 0.507\end{aligned}$$

Bias b

$$\begin{aligned}0.800 &- 0.1 \times (-0.0346) \\&= 0.803\end{aligned}$$

General formula

$$\begin{aligned}w_{\text{new}} &= w_{\text{old}} - \text{lr} \times dL/dw \\ \text{lr} &= \text{learning rate (e.g. 0.1)}\end{aligned}$$

Chain Rule (the secret)

If A → B → C, then:

$$dC/dA = dC/dB \times dB/dA$$

Multiply gradients at each step

PyTorch does ALL of backprop automatically:

<code>y_hat = model(x)</code>	# Step 1: forward pass
<code>loss = criterion(y_hat, y)</code>	# Step 2: compute loss
<code>loss.backward()</code>	# Steps 3-6: backprop AUTO
<code>optimizer.step()</code>	# Step 7: update weights

Activation derivatives

ReLU $d/dz = 1$ if $z > 0$, else 0

Sigmoid $d/dz = y_{\text{hat}}(1 - y_{\text{hat}})$

Tanh $d/dz = 1 - \tanh(z)^2$

Key insights to remember

- Negative gradient → increase weight to reduce loss
- Large gradient → weight has big influence on error
- Learning rate controls step size (too big = overshoot)
- Do this for EVERY weight in the network each iteration